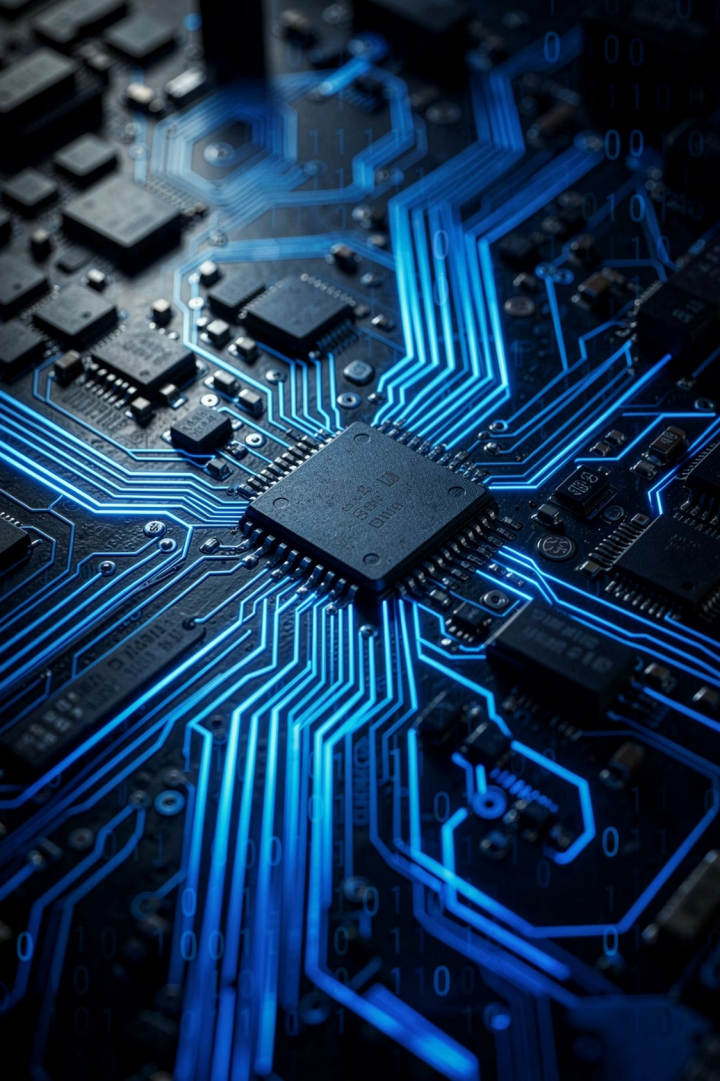




Diagramas en T y su Evolución

Integrantes:

Iván Hernández - C.I: 30.529.821

Miguel Gómez - C.I: 29.862.177

Miguel Barrios - C.I: 28.619.898

Daniel Valenzuela - C.I: 28.619.852



Procesamiento del Lenguaje Natural

El PLN es la disciplina que permite a las máquinas interpretar y generar lenguaje humano de forma valiosa. Cierra la brecha entre la lógica binaria de las computadoras y la complejidad ambigua del habla humana.

Análisis Multinivel

El sistema descompone oraciones a través de niveles morfológicos, léxicos y sintácticos.

Ejemplo Práctico

Al procesar "El ingeniero diseña el sistema", se realiza tokenización y etiquetado gramatical de sustantivos y verbos.

Aplicaciones

Asistentes virtuales, filtros de spam y sistemas como Google Translate.

El Surgimiento de la Era Simbólica

El Primer Gran Paradigma

La Era Simbólica sostenía que la inteligencia podía representarse mediante la manipulación de símbolos siguiendo reglas lógicas explícitas.

¿Cómo funcionaba?

El sistema **no aprendía de los datos**. Dependía totalmente de un conjunto exhaustivo de reglas escritas por expertos humanos: un auténtico "*manual gigante*" que debía anticipar cada caso posible del lenguaje.



Gramáticas Formales y Lógica de Reglas

El corazón de esta era fueron las **Gramáticas Formales**, impulsadas por lingüistas como **Noam Chomsky**. Las Gramáticas Libres de Contexto permitieron definir la estructura de los lenguajes naturales y de programación de manera matemática.

Lógica "Si-Entonces"

Los sistemas operaban bajo reglas de producción. Si se detectaba "Sustantivo + Verbo", la regla determinaba su traducción o ejecución.

Ejemplo de Traducción

Para traducir "Red car", una regla IF-THEN detectaba el adjetivo y el sustantivo e invertía su orden para producir "Carro rojo".

Diccionarios Estructurados

Vocabularios que almacenaban atributos morfológicos detallados para validar cada elemento de la oración.



Rigidez y Limitaciones Técnicas

Aunque eran sistemas transparentes y fáciles de depurar, su principal debilidad era la **rigidez**.

El Problema de la Ambigüedad

Un sistema rígido no podía decidir si "banco" era un asiento o una institución financiera sin contexto masivo.

Análisis Sintáctico Estricto

Si una oración no coincidía exactamente con las reglas predefinidas, el sistema se detenía y arrojaba un error. No existía inferencia por similitud.

Rendimiento Degradado

A medida que el vocabulario crecía, el motor tardaba más en encontrar un camino lógico, causando graves retrasos en traducciones largas.

El Rol de los Diagramas en T

Para diseñar y visualizar estos complejos sistemas surgieron los **Diagramas en T**, que representan la relación entre tres lenguajes en un proceso de traducción.



Esta representación visual permitía a los ingenieros razonar con claridad sobre la arquitectura de cualquier traductor o compilador, independientemente de los lenguajes involucrados.

Conclusión y Legado de la Era Simbólica

A pesar de sus limitaciones de adaptabilidad, la Era Simbólica dejó un legado invaluable para la informática moderna.

Formalización del Lenguaje

Obligó a formalizar matemáticamente el lenguaje humano, sentando bases teóricas rigurosas.

Cimientos para el Futuro

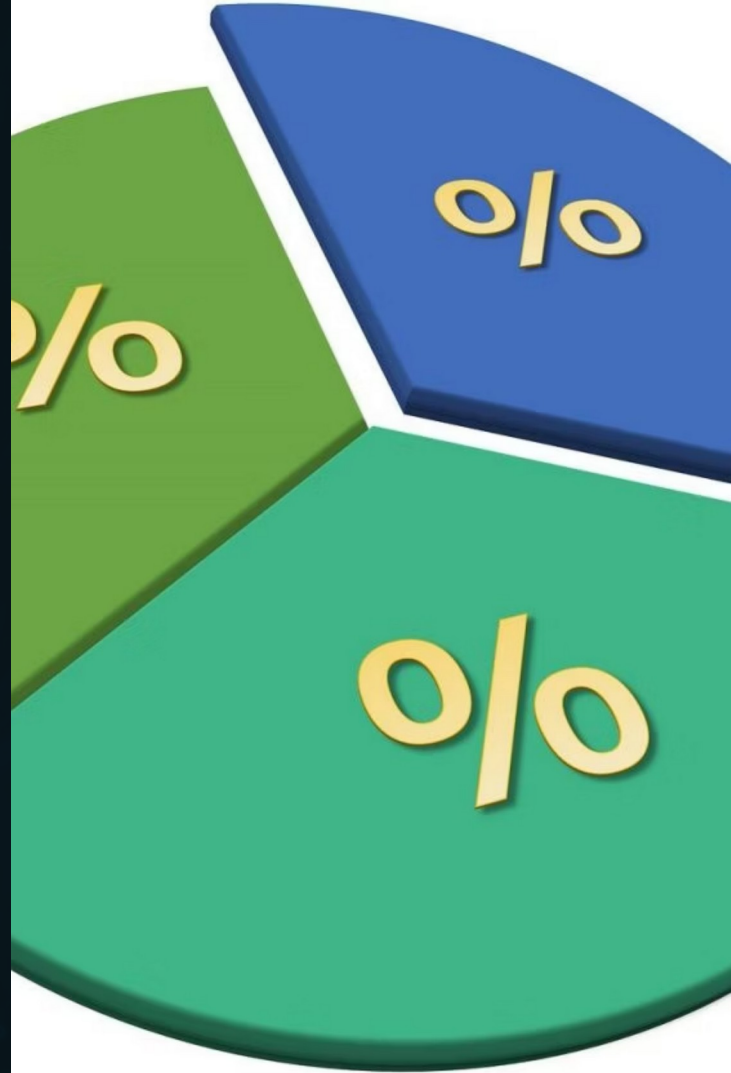
Sus fundamentos permiten hoy organizar bases de datos modernas y validar la coherencia gramatical en los sistemas actuales.



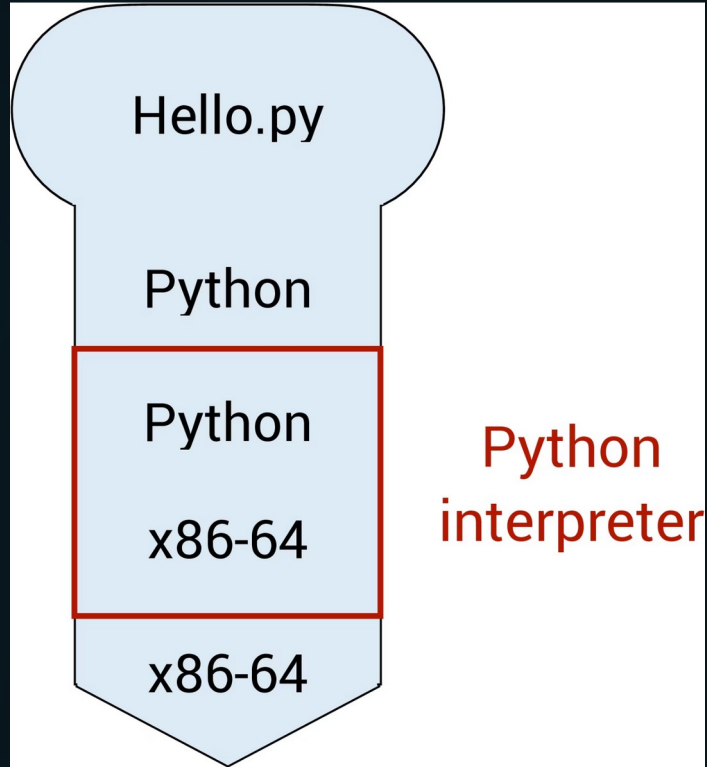
El Auge de la Probabilidad

Modelos estadísticos · 1990–2010

La "revolución estadística" transformó los diagramas T: de reglas puras y gramáticas formales a [modelos probabilísticos basados en datos](#), especialmente en traducción automática estadística (SMT).



Redefinición Probabilística del Diagrama T



Lenguaje Fuente (S)

Texto libre en idioma natural, no solo código fuente.

Lenguaje Objetivo (O)

Modelo de Canal Ruidoso con probabilidad condicionada.

Implementación (I)

Decodificador estadístico impulsado por corpus paralelos alineados.

Modelos Basados en Palabras · Años 90

Modelo 1 · Léxico Puro

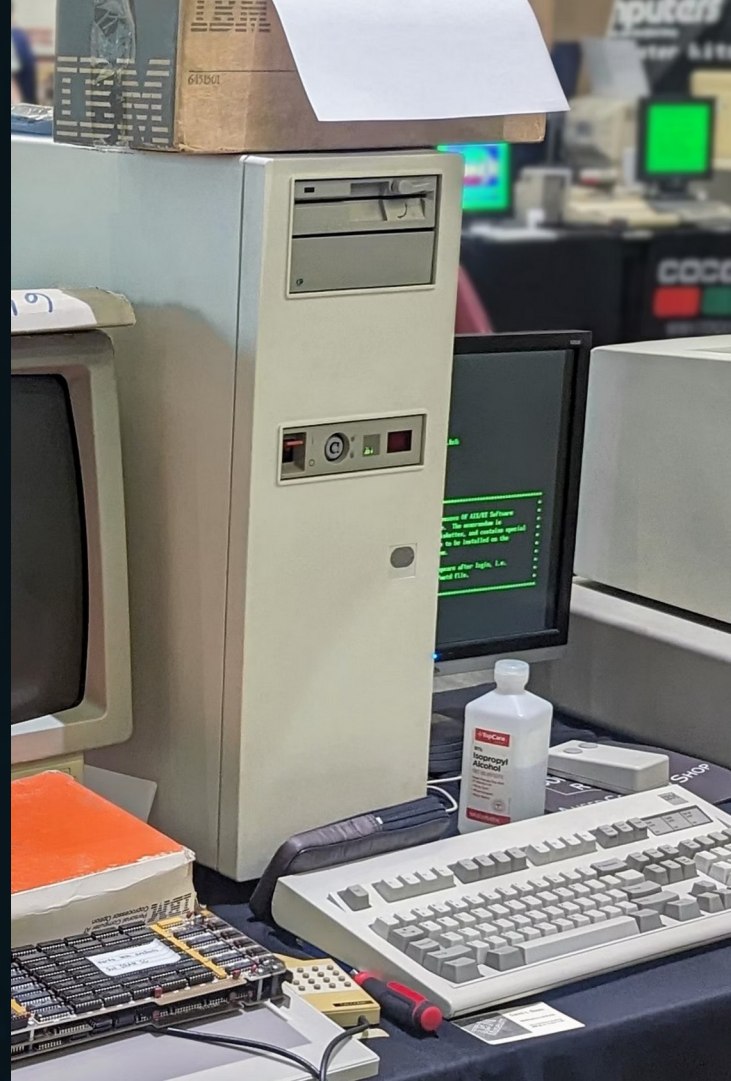
Probabilidad de que una palabra fuente se traduzca como otra en el destino.
Traducción palabra por palabra.

Modelo 2 · Posición

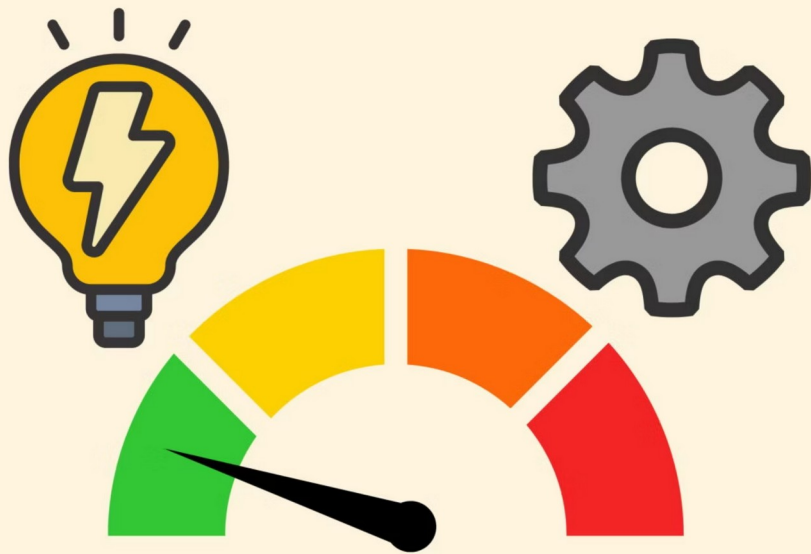
Considera la posición de las palabras en el idioma fuente para el destino.

Modelo 3 · Fertilidad

Una palabra fuente puede generar **2 o más palabras** en el idioma destino.



Modelos IBM restantes



Modelo 4 · Alineación relativa

Mejora el reordenamiento al usar posiciones relativas entre palabras y segmentos, capturando cambios de orden más complejos.

Modelo 5 · Deficiencia

Restringe alineaciones imposibles para mantener la coherencia estructural y producir traducciones válidas.

Modelos Basados en Frases · Años 2000



ESTÁNDAR DE LA INDUSTRIA · 2007

Auge de Moses

De palabra a palabra se pasó a **bloques de texto completos**. Sistema de código abierto que captura mejor el contexto inmediato.

Frase fuente (S) → Motor Moses → Frase destino (O)

Moses calcula la probabilidad de que un bloque se traduzca como otro bloque en el idioma destino, buscando fluidez y coherencia.

Modelos Basados en Sintaxis y Árboles

Finales de los 2000 · Gramáticas Libres de Contexto Probabilísticas (PCFG)

Árbol a Cadena

Se analiza la sintaxis fuente como árbol; el destino genera solo una cadena de texto.

Cadena a Árbol

Entrada en texto plano; el decodificador genera traducción y árbol sintáctico simultáneamente.

Árbol a Árbol

Entrada y salida representadas mediante árboles. Máxima complejidad estructural.

Los diagramas T visualizaban jerarquías gramaticales (sujeto, verbo...) y el reposicionamiento de palabras entre idiomas con estructuras muy diferentes, como inglés a japonés.

Impacto en la Práctica



Frontend



Backend

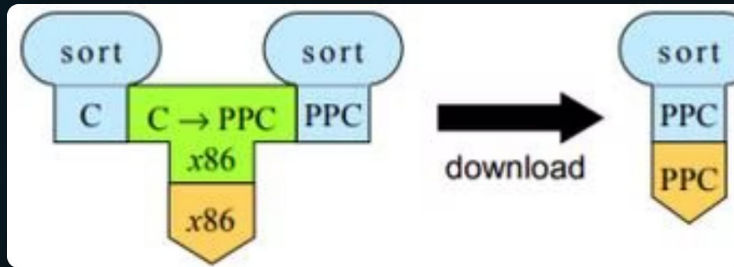
Bootstrapping Formalizado

Un subconjunto del lenguaje se usa para escribir su propio compilador.

Lenguajes Intermedios

División en **Front-end** (Source → Intermediate) y **Back-end** (Intermediate → Target). Ejemplo: P-Code de Pascal.

Impacto en la Práctica



Cross-Compilation

Los diagramas T definen cómo una máquina con arquitectura **X** genera código para una arquitectura **Y**.

Vital para **sistemas embebidos** y entornos con recursos limitados.

Impacto en la Práctica

Sin reglas manuales

Con suficientes datos empíricos (actas del Parlamento Europeo, ONU), el modelo aprendía a traducir automáticamente.

Arquitectura dual

Modelo de Traducción **P(S/O)** combinado con Modelo de Lenguaje **P(O)**.

Dominó hasta 2010

Esta arquitectura fue el estándar hasta la llegada de las redes neuronales profundas.



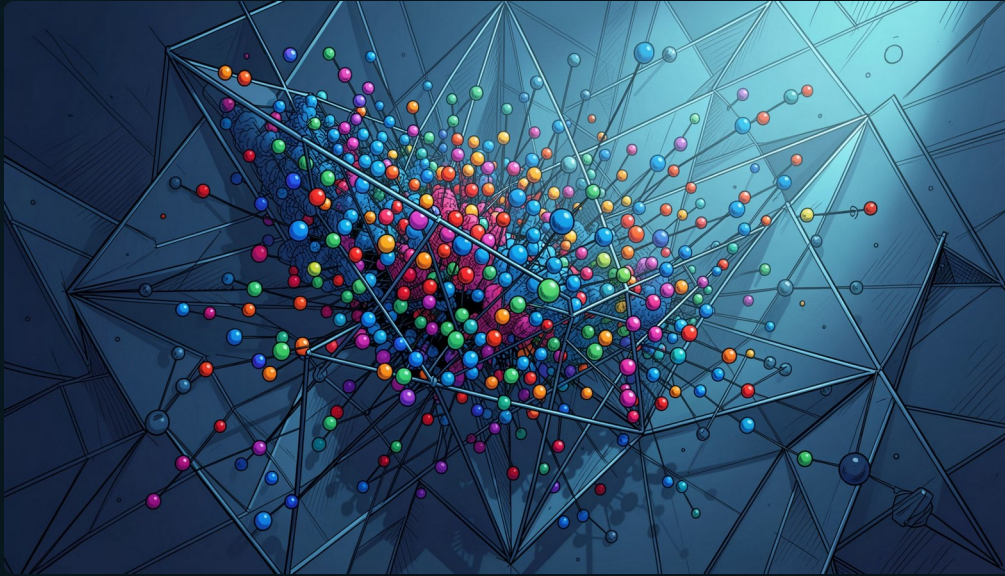


¿Qué es el Embedding?

El embedding representa objetos —texto, imágenes, audio— como puntos en un **espacio vectorial continuo** donde las posiciones son semánticamente significativas para algoritmos de ML. Permite a los modelos encontrar objetos similares sin requerir experiencia humana para definirlos: los patrones se aprenden automáticamente de los datos.

MOTORES DE BÚSQUEDA · RECOMENDACIÓN · CHATBOTS · DETECCIÓN DE FRAUDE

¿Cómo funciona el Embedding?



Los algoritmos de ML requieren datos numéricos. Los objetos se convierten en **vectores**: arreglos de números donde cada dimensión representa una característica del objeto.

Vector embedding

Arreglo numérico (ej. 1489, 22... 3, 777) con cientos o miles de dimensiones.

Similitud semántica

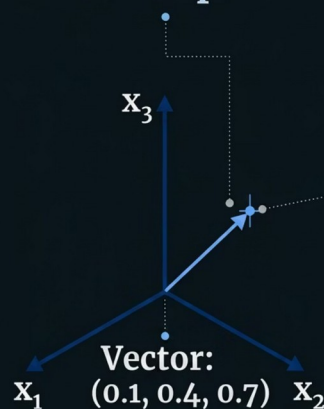
Cuanto más cerca estén dos embeddings en el espacio n-dimensional, más similares son.

Word2Vec

Desarrollado por **Google en 2013**, Word2Vec crea embeddings de palabras mediante una red neuronal de dos capas. Al graficar los vectores en un espacio tridimensional, **los sinónimos se agrupan** naturalmente.

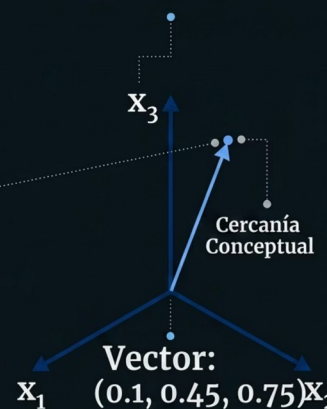
❗ **Ejemplo:** "papá" y "mamá" son similares, pero "padre" estaría mucho más cerca de "papá" en el espacio vectorial, produciendo un producto punto más alto.

Representación
de "Papá"



Representación
de "Papá"

Representación
de "Mamá"



Representación
de "Mamá"

Cercanía
Conceptual

¿Cómo se crean los Embeddings?

01

Elegir o entrenar el modelo

Seleccionar Word2Vec, GloVe, BERT u otro según la tarea.

02

Preparar los datos

Tokenizar texto, redimensionar y normalizar imágenes.

03

Cargar o entrenar el modelo

Cargar pesos preentrenados o entrenar desde cero.

04

Generar embeddings

Obtener el vector correspondiente para cada punto de datos.

05

Integrar en la aplicación

Usar para búsqueda por similitud, recomendación, clustering, etc.

Embeddings en PLN

El PLN evolucionó de un enfoque **simbólico y basado en reglas** hacia uno **probabilístico y neuronal**, donde el contexto y la semántica son lo más importante.

Análisis de sentimientos

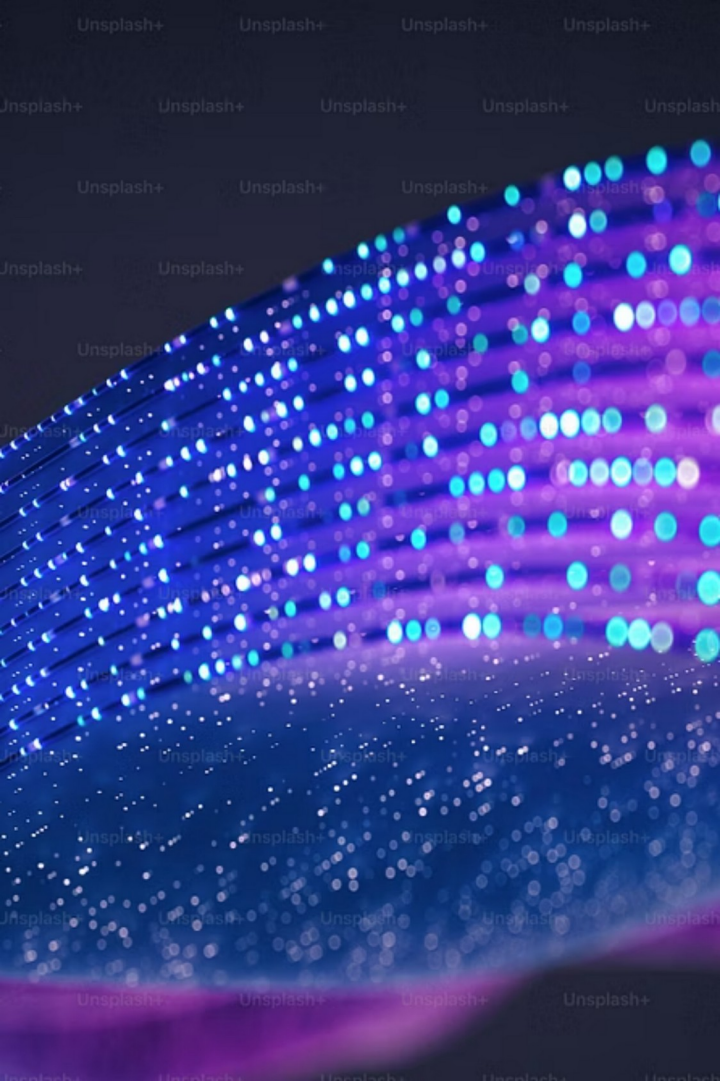
Word2Vec y GloVe representan palabras para clasificar el sentimiento de un texto.

Preguntas y respuestas

BERT comprende el contexto de preguntas y documentos para extraer información relevante.

Similitud de documentos

Doc2Vec compara embeddings de documentos para medir similitud semántica.



La Era de la Atención y los Transformers

2018 – Presente: Un cambio de paradigma total en el procesamiento de lenguaje natural (NLP) y la teoría de compiladores. Las arquitecturas secuenciales como RNN y LSTM cedieron el paso a los [Transformers](#), redefiniendo tanto el lenguaje humano como la ingeniería de software.

El Hito Fundacional: *Attention is All You Need*

Publicado por Google en 2017, su impacto real transformó la industria a partir de 2018.

Autoatención

El Transformer observa toda la secuencia simultáneamente, no token por token.

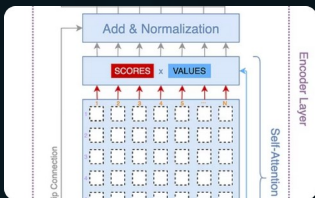
Paralelismo

Elimina los "pasos temporales" de las RNN, permitiendo entrenar modelos masivos en GPUs.

Contexto Global

Captura relaciones entre palabras o variables muy alejadas, resolviendo dependencias a largo plazo.

Evolución de Modelos Transformer



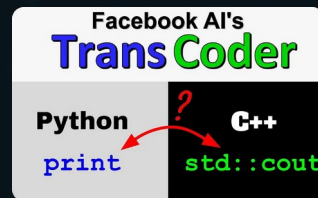
Codificadores (Encoder-only)

BERT (2018). Análisis sintáctico, detección de errores y búsqueda semántica de código.



Decodificadores (Decoder-only)

Familia GPT. Generación de código; base de GitHub Copilot para autocompletar funciones enteras.



Encoder-Decoder

T5 y BART. Traducción y transpilación: convierten código entre lenguajes (ej. Python a C++).

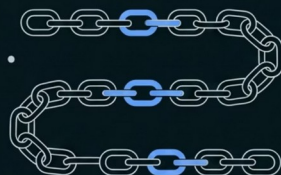
Transformers en Compiladores

El Transformer como Compilador

Investigaciones recientes demuestran que los Transformers emulan tareas críticas de compilación:

- **Construcción de AST:** Aprenden estructura jerárquica sin reglas gramaticales explícitas.
- **Resolución de tipos:** Infieren tipos y scope analizando contexto de uso.
- **Optimización de código:** LLVM y GCC integran modelos de atención para decidir optimizaciones (unrolling, asignación de registros) en lugar de heurísticas fijas.

ERA PRE-2018 (RNN/LSTM)



Procesamiento Secuencial

Contexto:
Gradiente desvaneciente

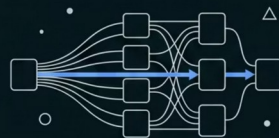
Gradiente Desvaneciente

Contexto Limitado

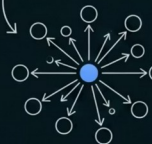


Embeddings Estáticos
Word2Vec

ERA DE LA ATENCIÓN (TRANSFORMERS)



Procesamiento Paralelo



Self-Attention:
Acceso Directo
a Tokens

Embeddings Contextuales
BERT/GPT



Genera y Optimiza Lógica

Estado Actual y Futuro

Vivimos la era de los **LLMs aplicados al código**: Codex (OpenAI) y CodeT5 (Salesforce) lideran la tendencia.

Multimodalidad

Modelos que comprenden tanto documentación en lenguaje natural como código ejecutable.

Compiladores Híbridos

Frontend simbólico y formal para garantizar corrección; backend de optimización manejado por agentes de IA basados en atención.

